人体识别报警追踪

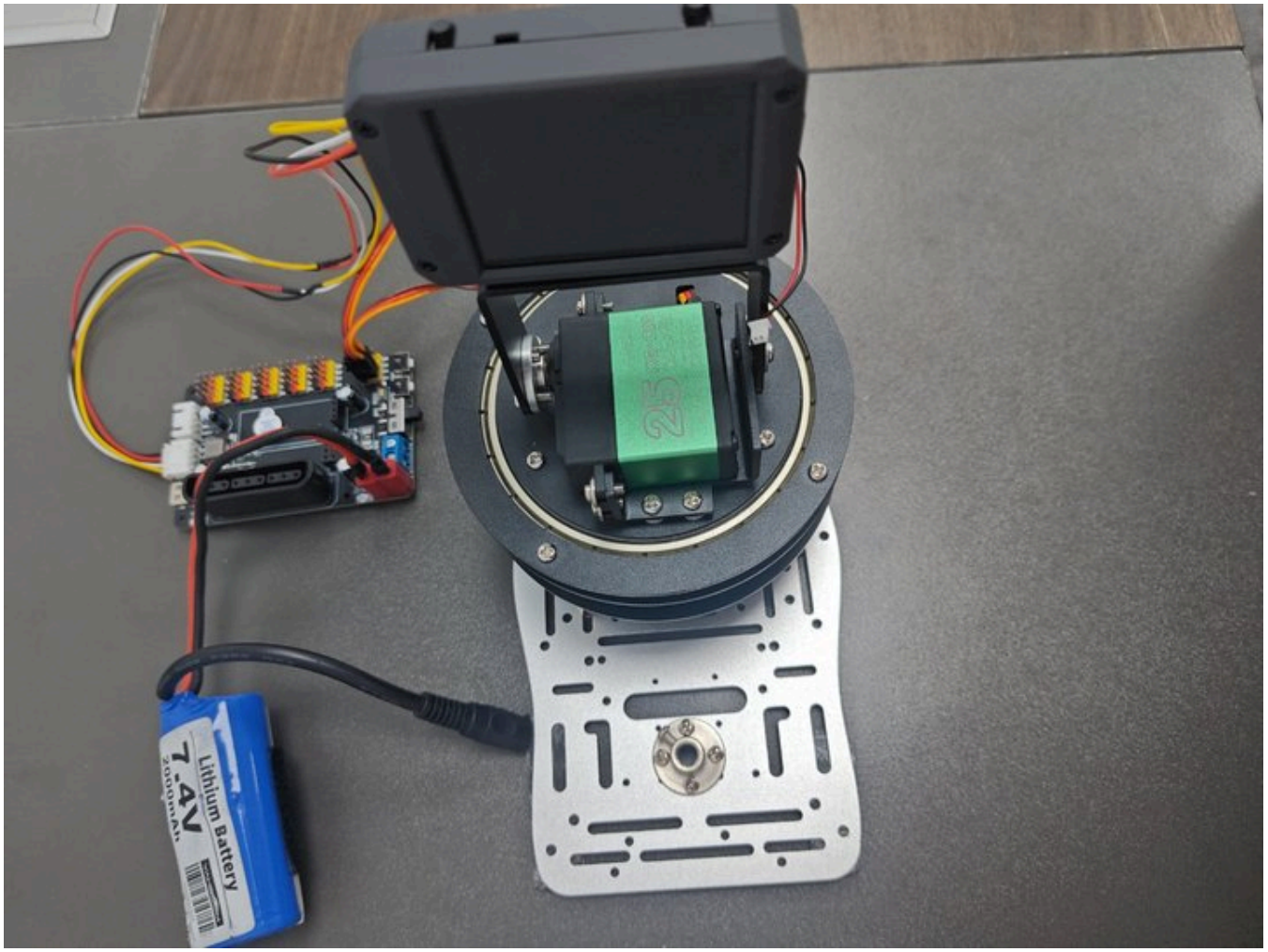
1.开篇说明

请先阅读二十四路舵机驱动板以及stm32入门教程里面的资料，了解清楚如何烧录程序、入门stm32的基础。这样方便学习和烧录后面的程序。

2.器材清单

- k230 x1
- 24路舵机驱动板 x1
- 7.4v电池 x1 购买
- 铝合金板 x1 购买

3.摆放位置

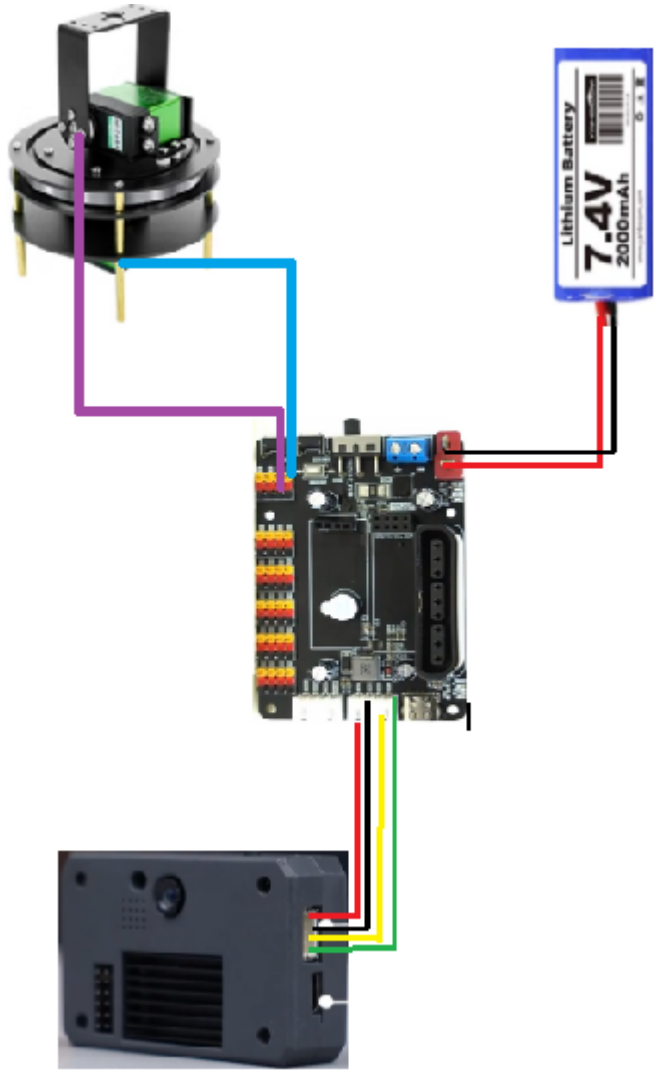


摆放位置如图所示，可以选择我们的铝基板车架作为固定k230和云台的底板，也可以自行选择能固定的底板，底盘可以胶给粘住以免云台动的幅度大导致偏移，k230固定在二维云台上面,用电池的时候如果云台转动幅度过大可以适度减小pid参数。

4.实验准备

首先按照二维云台安装教程搭建好二维云台

硬件接线：



二维云台	24路舵机驱动板
X轴舵机	S1
Y轴舵机	S2

K230视觉模块	24路电机驱动板
5V	5V
GND	GND
TX	RX3
RX	TX3

注意:插舵机的线时一定要将黄色线对准24路舵机驱动板的黄色GPIO口，褐色线对准24路舵机驱动板的黑色GPIO口。

5.K230关键代码解析

代码结构

主程序流程：

- 初始化PipeLine
- 创建PipeLine实例
- 执行Demo函数

Demo执行流程：

- 初始化参数（模型路径、阈值等）
- 初始化人体检测类
- 进入主循环处理图像

人体检测类功能：

- 类初始化（设置参数和AI2D）
- 预处理流程（padding和resize）
- 后处理流程（处理检测结果）
- 结果绘制（绘制检测框和标签）
- 数据传输 (将得到坐标数据给stm32)

异常处理：

- 捕获异常并打印信息
- 执行反初始化清理

代码片段

```
from libs.PipeLine import PipeLine, ScopedTiming
from libs.AIBase import AIBase
from libs.AI2D import Ai2d
from ybUtils.YbBuzzer import YbBuzzer
from libs.YbProtocol import YbProtocol
from ybUtils.YbUart import YbUart
import os
import ujson
from media.media import *
from time import *
import nncase_runtime as nn
import ulab.numpy as np
import time
import utime
import image
import random
import gc
import sys
import aicube
```

```
person_det = None


buzzer = YbBuzzer()

last_cx = 0
last_cy = 0
# uart = None
uart = YbUart(baudrate=115200)
pto = YbProtocol()

def conversion_postion(x,y):
    if x > 510:
       x_h = 255
       x_l = 255
       x_z = x-255-255
    elif 510>=x>=255:
       x_h = 255
       x_l = x-255
       x_z = 0
    else:
       x_h = x
       x_l = 0
       x_z = 0
    if y > 510:
       y_h = 255
       y_l = 255
       y_z = y-255-255
    elif 510>=y>=255:
       y_h = 255
       y_l = y-255
       y_z = 0
    else:
       y_h = y
       y_l = 0
       y_z = 0
    return x_h,x_l,x_z,y_h,y_l,y_z
# 自定义人体检测类
# (Custom person detection class)
class PersonDetectionApp(AIBase):
    def __init__(self, kmodel_path, model_input_size, labels, anchors,
confidence_threshold=0.2, nms_threshold=0.5, nms_option=False, strides=[8, 16, 32],
rgb888p_size=[224, 224], display_size=[640, 360], debug_mode=0):
        """
        初始化人体检测应用类
        (Initialize the person detection application class)

        参数:
        kmodel_path: 模型文件路径
        (kmodel_path: Path to the model file)
        model_input_size: 模型输入分辨率
        (model_input_size: Model input resolution)
```

```
        labels: 标签列表
        (labels: List of labels)
        anchors: 检测anchors设置
        (anchors: Detection anchor settings)
        confidence_threshold: 置信度阈值
        (confidence_threshold: Confidence threshold)
        nms_threshold: nms阈值
        (nms_threshold: NMS threshold)
        nms_option: nms是否开启
        (nms_option: Whether NMS is enabled)
        strides: 特征图降采样倍数
        (strides: Feature map downsampling factor)
        rgb888p_size: 传感器输入图像分辨率
        (rgb888p_size: Sensor input image resolution)
        display_size: 显示分辨率
        (display_size: Display resolution)
        debug_mode: 调试模式级别
        (debug_mode: Debug mode level)
        """
        super().__init__(kmodel_path, model_input_size, rgb888p_size, debug_mode)
        self.kmodel_path = kmodel_path
        # 模型输入分辨率
        # (Model input resolution)
        self.model_input_size = model_input_size
        # 标签
        # (Labels)
        self.labels = labels
        # 检测anchors设置
        # (Detection anchor settings)
        self.anchors = anchors
        # 特征图降采样倍数
        # (Feature map downsampling factor)
        self.strides = strides
        # 置信度阈值设置
        # (Confidence threshold setting)
        self.confidence_threshold = confidence_threshold
        # nms阈值设置
        # (NMS threshold setting)
        self.nms_threshold = nms_threshold
        self.nms_option = nms_option
        # sensor给到AI的图像分辨率
        # (Image resolution from sensor to AI)
        self.rgb888p_size = [ALIGN_UP(rgb888p_size[0], 16), rgb888p_size[1]]
        # 显示分辨率
        # (Display resolution)
        self.display_size = [ALIGN_UP(display_size[0], 16), display_size[1]]
        self.debug_mode = debug_mode
        # Ai2d实例，用于实现模型预处理
        # (Ai2d instance for model preprocessing)
        self.ai2d = Ai2d(debug_mode)
        # 设置Ai2d的输入输出格式和类型
        # (Set Ai2d input and output formats and types)
```

```
        self.ai2d.set_ai2d_dtype(nn.ai2d_format.NCHW_FMT, nn.ai2d_format.NCHW_FMT, np.uint8,
np.uint8)

    # 配置预处理操作，这里使用了pad和resize，Ai2d支持crop/shift/pad/resize/affine，具体代码请打
开/sdcard/app/libs/AI2D.py查看
    # (Configure preprocessing operations, using pad and resize here, Ai2d supports
crop/shift/pad/resize/affine)
    def config_preprocess(self, input_image_size=None):
        """
        配置模型输入数据的预处理操作
        (Configure preprocessing operations for model input data)

        参数:
        input_image_size: 输入图像尺寸，如果为None则使用默认rgb888p_size
        (input_image_size: Input image size, if None uses default rgb888p_size)
        """
        with ScopedTiming("set preprocess config", self.debug_mode > 0):
            # 初始化ai2d预处理配置，默认为sensor给到AI的尺寸，您可以通过设置input_image_size自行修改输
入尺寸
            # (Initialize ai2d preprocessing configuration, default is the size from sensor
to AI)
            ai2d_input_size = input_image_size if input_image_size else self.rgb888p_size
            top, bottom, left, right = self.get_padding_param()
            self.ai2d.pad([0, 0, 0, 0, top, bottom, left, right], 0, [0, 0, 0])
            self.ai2d.resize(nn.interp_method.tf_bilinear, nn.interp_mode.half_pixel)
            self.ai2d.build([1, 3, ai2d_input_size[1], ai2d_input_size[0]], [1, 3,
self.model_input_size[1], self.model_input_size[0]])

    # 自定义当前任务的后处理
    # (Custom post-processing for the current task)
    def postprocess(self, results):
        """
        对模型输出进行后处理
        (Post-process the model output)

        参数:
        results: 模型的原始输出
        (results: Raw output from the model)

        返回:
        处理后的检测结果
        (Processed detection results)
        """
        with ScopedTiming("postprocess", self.debug_mode > 0):
            # 这里使用了aicube模型的后处理接口anchorbasedet_post_preocess
            # (Use the anchorbasedet_post_process interface from the aicube model for post-
processing)
            dets = aicube.anchorbasedet_post_process(results[0], results[1], results[2],
self.model_input_size, self.rgb888p_size, self.strides, len(self.labels),
self.confidence_threshold, self.nms_threshold, self.anchors, self.nms_option)
            return dets

    # 绘制结果
```

```
    # (Draw results)
    def draw_result(self, pl, dets):

        """
        在画布上绘制人体检测结果
        (Draw person detection results on the canvas)

        参数:
        pl: 画布对象
        (pl: Canvas object)
        dets: 检测结果
        (dets: Detection results)
        """
        with ScopedTiming("display_draw", self.debug_mode > 0):
            if dets:
                pl.osd_img.clear()
                for det_box in dets:
                    # 计算检测框在显示分辨率下的坐标和尺寸
                    # (Calculate the coordinates and size of the detection box in the
display resolution)
                    x1, y1, x2, y2 = det_box[2], det_box[3], det_box[4], det_box[5]
                    w = float(x2 - x1) * self.display_size[0] // self.rgb888p_size[0]
                    h = float(y2 - y1) * self.display_size[1] // self.rgb888p_size[1]
                    x1 = int(x1 * self.display_size[0] // self.rgb888p_size[0])
                    y1 = int(y1 * self.display_size[1] // self.rgb888p_size[1])
                    x2 = int(x2 * self.display_size[0] // self.rgb888p_size[0])
                    y2 = int(y2 * self.display_size[1] // self.rgb888p_size[1])

                    # 过滤掉尺寸太小的检测框
                    # (Filter out detection boxes that are too small)
                    if (h < (0.1 * self.display_size[0])):
                        continue
                    if (w < (0.25 * self.display_size[0]) and ((x1 < (0.03 *
self.display_size[0])) or (x2 > (0.97 * self.display_size[0])))):
                        continue
                    if (w < (0.15 * self.display_size[0]) and ((x1 < (0.01 *
self.display_size[0])) or (x2 > (0.99 * self.display_size[0])))):
                        continue

                    # 在画布上绘制检测框和标签
                    # (Draw the detection box and label on the canvas)
                    pl.osd_img.draw_rectangle(x1, y1, int(w), int(h), color=(255, 0, 255,
0), thickness=2)
#                     uart.send(f"${x1},{y1},{w},{h}#")
                    pl.osd_img.draw_string_advanced(x1, y1 - 50, 32, " " +
self.labels[det_box[0]] + " " + str(round(det_box[1], 2)), color=(255, 0, 255, 0))
                    #pto_data = pto.get_person_detect_data(x1, y1, w, h)
                    cx_h,cx_l,cx_z,cy_h,cy_l,cy_z=conversion_postion(x1,y1)

                    buzzer.beep()
                    ]

 uart.write(bytearray([0x55,0xaa,0x00,int(cx_h),int(cx_l),int(cx_z),0xfa]))
```

```
 uart.write(bytearray([0x55,0xaa,0xff,int(cy_h),int(cy_l),int(cy_z),0xfa]))


            else:
                pl.osd_img.clear()

    # 计算padding参数
    # (Calculate padding parameters)
    def get_padding_param(self):
        """
        计算将输入图像等比缩放到模型输入尺寸时需要的padding参数
        (Calculate the padding parameters required to proportionally scale the input image
to the model input size)

        返回:
        padding参数 [top, bottom, left, right]
        (Padding parameters [top, bottom, left, right])
        """
        dst_w = self.model_input_size[0]
        dst_h = self.model_input_size[1]
        input_width = self.rgb888p_size[0]
        input_high = self.rgb888p_size[1]
        ratio_w = dst_w / input_width
        ratio_h = dst_h / input_high
        if ratio_w < ratio_h:
            ratio = ratio_w
        else:
            ratio = ratio_h
        new_w = (int)(ratio * input_width)
        new_h = (int)(ratio * input_high)
        dw = (dst_w - new_w) / 2
        dh = (dst_h - new_h) / 2
        top = int(round(dh - 0.1))
        bottom = int(round(dh + 0.1))
        left = int(round(dw - 0.1))
        right = int(round(dw - 0.1))
        return top, bottom, left, right


def exce_demo(pl):
    """
    执行人体检测演示
    (Execute person detection demo)

    参数:
    pl: PipeLine对象
    (pl: PipeLine object)
    """
    global person_det

    display_mode = pl.display_mode
    rgb888p_size = pl.rgb888p_size
```

```
    display_size = pl.display_size

    # 设置模型路径和其他参数
    # (Set model path and other parameters)
    kmodel_path = "/sdcard/kmodel/person_detect_yolov5n.kmodel"
    # 其它参数设置
    # (Other parameter settings)
    confidence_threshold = 0.7
    nms_threshold = 0.7
    labels = ["person"]
    anchors = [10, 13, 16, 30, 33, 23, 30, 61, 62, 45, 59, 119, 116, 90, 156, 198, 373,
326]

    try:
        # 初始化自定义人体检测实例
        # (Initialize the custom person detection instance)
        person_det = PersonDetectionApp(kmodel_path, model_input_size=[640, 640],
labels=labels, anchors=anchors, confidence_threshold=confidence_threshold,
nms_threshold=nms_threshold, nms_option=False, strides=[8, 16, 32],
rgb888p_size=rgb888p_size, display_size=display_size, debug_mode=0)
        person_det.config_preprocess()
        while True:
            with ScopedTiming("total", 1):
                # 获取当前帧数据
                # (Get the current frame data)
                #•buzzer = YbBuzzer()
                img = pl.get_frame()
                # 推理当前帧
                # (Inference on the current frame)
                res = person_det.run(img)
                # 绘制结果到PipeLine的osd图像
                # (Draw the results on the PipeLine's osd image)
                person_det.draw_result(pl, res)

                # 显示当前的绘制结果
                # (Display the current drawing results)
                pl.show_image()
                gc.collect()
    #except Exception as e:
     #   print("人体检测功能退出")
        # (Person detection function exited)
    finally:
        person_det.deinit()                          # 反初始化
        # (Deinitialize)

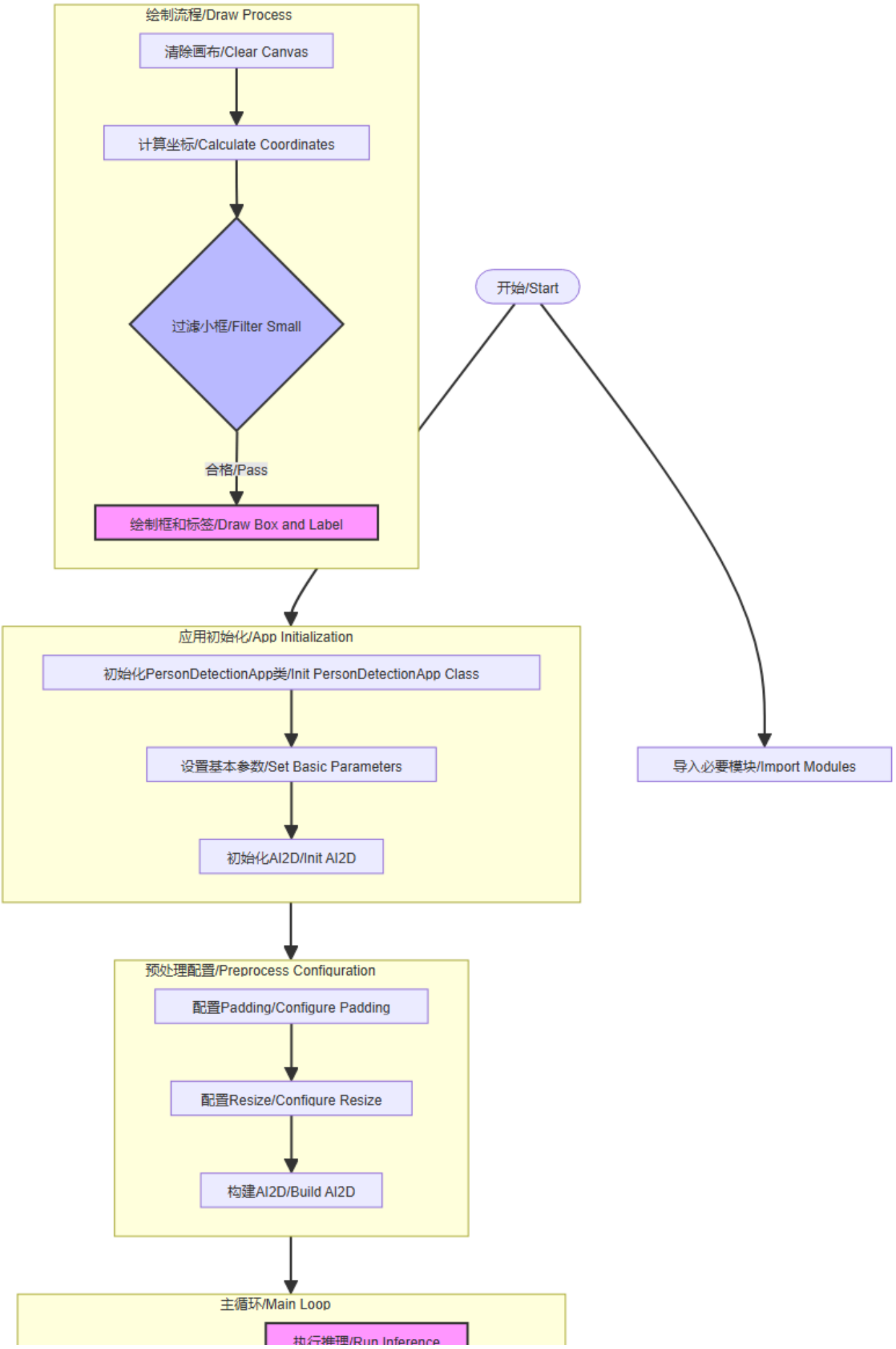
def exit_demo():
    """
    退出人体检测演示
    (Exit person detection demo)
    """
    global person_det
    person_det.deinit()
```

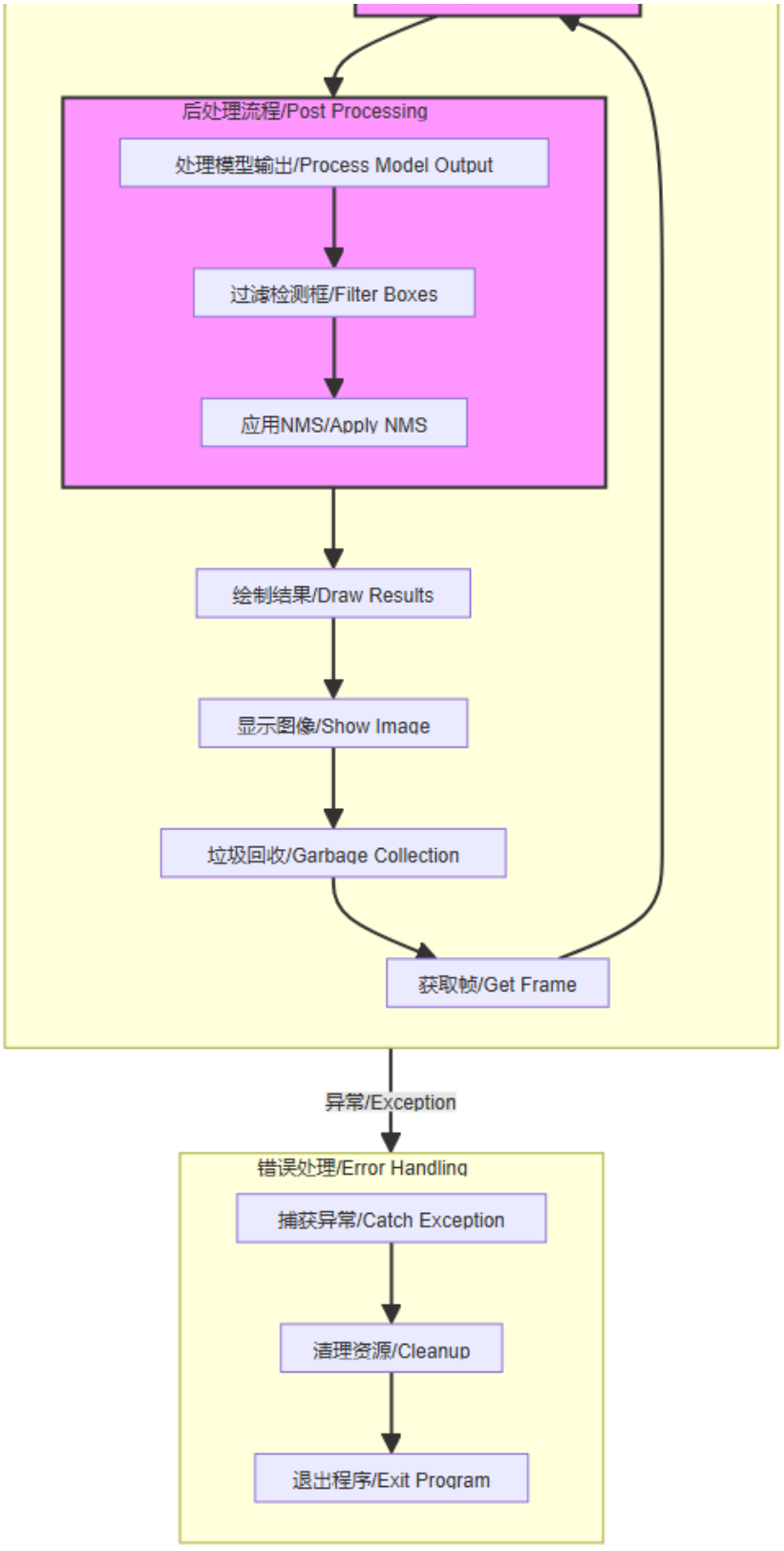
```
if __name__ == "__main__":

    rgb888p_size = [640, 360]
    display_size = [640, 480]
    display_mode = "lcd"

    # 初始化PipeLine，用于图像处理流程
    # (Initialize PipeLine for image processing)
    pl = PipeLine(rgb888p_size=rgb888p_size, display_size=display_size,
display_mode=display_mode)
    pl.create()  # 创建PipeLine实例
    # (Create PipeLine instance)
    # 初始化自定义人脸检测实例
    # (Initialize the custom person detection instance)
    exce_demo(pl)
```

流程图


绘制流程/Draw Process
清除画布/Clear Canvas
计算坐标/Calculate Coordinates
过滤小框/Filter Small
合格/Pass
绘制框和标签/Draw Box and Label
开始/Start
导入必要模块/Import Modules
应用初始化/App Initialization
初始化PersonDetectionApp类/Init PersonDetectionApp Class
设置基本参数/Set Basic Parameters
初始化AI2D/Init AI2D
预处理配置/Preprocess Configuration
配置Padding/Configure Padding
配置Resize/Configure Resize
构建AI2D/Build AI2D
主循环/Main Loop


后处理流程/Post Processing
处理模型输出/Process Model Output
过滤检测框/Filter Boxes
应用NMS/Apply NMS
绘制结果/Draw Results
显示图像/Show Image
垃圾回收/Garbage Collection
获取帧/Get Frame
异常/Exception
错误处理/Error Handling
捕获异常/Catch Exception
清理资源/Cleanup
退出程序/Exit Program

6.stm32关键代码解析

- **control.c**

```
uint8_t get_point2_p()   //处理坐标函数
{
    uint8_t lable, load_1, load_2 ,load_3 ,ret=0;
    if(DataDecode1(&Uart3_RingBuff,&lable, &load_1, &load_2,&load_3) == 0){
     if(lable == 0x00){
            rec_step_p[0][0] = load_1 + load_2 + load_3 ;
//          ret=1;

        } else if(lable == 0xff){

            rec_step_p[0][1] = load_1 + load_2 + load_3 ;
//          ret=1;

        }
```

```
    return ret;
}
}
```

此代码是将k230传过的3帧数据加在一起，由于前面说过将k230的坐标数据拆成3帧数据然后分别发送，这里正是将数据加在一起这样形成正确的坐标，将坐标赋值给二维数组。

```
    get_point2_p();

    set_target(rec_step_p[0][0],rec_step_p[0][1]);
```

此代码是在上面获得人脸坐标的基础上将坐标作为pid目标值，然后控制舵机运动。

- **bsp_servo.c**

```
ServTypdef Xserv = {
    .pwm = 2000,
    .MAXPWM = 2500,
    .MINPWM = 500,
    .MIDPWM = 1400
};

ServTypdef Yserv = {
    .pwm = 1000,
    .MAXPWM = 2300,
    .MINPWM = 500,
    .MIDPWM = 1000
};
```

此代码是设置舵机启动的起始位置，所以需要修改这个来拟合云台运动的顶点，我将云台开机初始位置设置在右上角，然后云台可以通过k230的坐标变化来按比例进行调整，如果在调整云台位置和细调位置的时候就需要调这个位置参数了。

```
void servo_ctr(ServTypdef *servo, int PidInput)
{
    uint16_t CompSet = servo->MIDPWM + PidInput;

    if( CompSet > servo->MAXPWM ){
        CompSet = servo->MAXPWM;
    } else if ( CompSet < servo->MINPWM  ){
        CompSet = servo->MINPWM;
    }
    if(servo->pwm == CompSet){/*避免重复更改比较值*/

        return;
    }
```

```
    if( servo == &Xserv ){


    GPIO_SetBits(Servo_J1_PORT, Servo_J1_PIN );      //将舵机电平置高
    delay_us(CompSet);                    //延时脉冲值的微秒数

    GPIO_ResetBits(Servo_J1_PORT, Servo_J1_PIN );            //将舵机电平置低
    delay_ms(20 - CompSet/1000);
    }
    else if( servo == &Yserv ){


    GPIO_SetBits(Servo_J2_PORT, Servo_J2_PIN );      //将舵机电平置高
    delay_us(CompSet);                         //延时脉冲值的微秒数

    GPIO_ResetBits(Servo_J2_PORT, Servo_J2_PIN );            //将舵机电平置低
    delay_ms(20 - CompSet/1000);
    }
}
```

此代码是将pid的输出给pwm，是将pid与pwm连接的地方，同时给pwm进行限幅，也是舵机运动的核心代码，通过延时和pwm的改变来控制舵机转动，将舵机运动和pwm联系一起。

- **pid.c**

```
float pid_calculate(PID_TypeDef *PID,float CurrentValue)
{
  PID->Err =  PID->Target - CurrentValue;
    PID->Integral += PID->Err;
    if(PID->Integral > 7000){
        PID->Integral = 7000;
    }
    if(PID->Integral < -7000){
        PID->Integral = -7000;
    }    //积分限幅
    PID->PID_out = PID->Kp * PID->Err                                    /*比例*/
         + PID->Ki * PID->Integral                                  /*积分*/
             + PID->Kd * (PID->Err - PID->LastErr);                      /*微分*/

    PID->LastErr = PID->Err;
        return PID->PID_out;
}
```

此代码是位置式pid运算的配置，包括积分限幅，位置式PID是通过计算当前偏差值（目标值与实际值之差）、积分值（过去误差的总和）和微分值（误差变化率），然后将这三个部分加权相加得到PID输出，如果想微调代码想达到更好的效果可以微调主函数的pid参数。

- **bsp_usart.c**

```
uint8_t DataDecode1(RingBuff_t *ringbuff, uint8_t *data1, uint8_t *data2, uint8_t
*data3,uint8_t *data4)
{
    static uint8_t uart_dec_count;
    static uint8_t uart_rec_data[6];
    uint8_t ret = 1;

    if(Read_RingBuff(ringbuff, &uart_rec_data[uart_dec_count]) == RINGBUFF_ERR){
        return 1;

    }
        if((uart_dec_count == 0)&&(uart_rec_data[uart_dec_count] != 0x55)) {             //帧
头0x55
        uart_rec_data[uart_dec_count] = 0;
    }
        else if((uart_dec_count == 1)&&(uart_rec_data[uart_dec_count] != 0xaa)){       //第二
帧0xaa
        uart_rec_data[uart_dec_count] = 0;
        uart_rec_data[uart_dec_count-1] = 0;
        uart_dec_count = 0;
        }else if((uart_dec_count == 6)&&(uart_rec_data[uart_dec_count] != 0xfa)){       //帧
尾0xfa
        uart_rec_data[uart_dec_count] = 0;
        uart_rec_data[uart_dec_count-1] = 0;
        uart_rec_data[uart_dec_count-2] = 0;
        uart_rec_data[uart_dec_count-3] = 0;
        uart_rec_data[uart_dec_count-4] = 0;
        uart_rec_data[uart_dec_count-5] = 0;
        uart_dec_count = 0;
    }
    else{
        if(uart_dec_count == 6)//成功接收一帧数据
        {
            *data1 = uart_rec_data[2];
        *data2 = uart_rec_data[3];
                *data3 = uart_rec_data[4];
            *data4 = uart_rec_data[5];
        ret = 0;

        }
        uart_dec_count++;
        if(uart_dec_count == 7)
        {
            uart_dec_count = 0;
        }

    }
    return ret;

}
```

此代码是接收k230数据的代码，在识别到帧头为0x55，第二个数据为0xaa，帧尾为0xfa的时候才接收数据，通过三个数据定位一帧数据，由于从k230传输过来的4个顶点坐标每个顶点又通过拆分成3组数据传输过来，所以传输过来的数据是非常庞大的，为了确保传过来的数据是准确的，所以采用三个数据来确认一帧数据。

7.效果描述

k230定义人体特征，当k230通过人体特征识别到人体的时候发出警报，然后将人体坐标传输给二维云台，二维云台通过坐标跟踪到人体，人移动二维云台也移动，达到追踪的效果。